

STIEditor User Manual

STIEditor Application Notes
May 5, 2021

Advantest Europe GmbH
Branch / Zweigniederlassung Boeblingen
Herrenberger Str. 130
D-71034 Boeblingen

Contents

1	Introduction	3
1.1	Audience	3
1.2	Terms Used in This Manual	3
1.3	Concept	3
2	Using STIEditor	5
2.1	Synopsis	5
2.2	Command Line Options Description	5
3	Usage Tutorial	7
3.1	Tutorial Goals	7
3.2	Editing Tutorial	7
3.2.1	Goals	7
3.2.2	Inputs	7
3.2.3	Setup	9
3.2.4	Running STIEditor	9
3.2.5	Results	9
3.3	Pattern Merge - Serial Mode	11
3.3.1	Goal	11
3.3.2	Inputs	12
3.3.3	Setup	12
3.3.4	Running STIEditor	12
3.3.5	Results	12
3.4	Pattern Merge - Parallel Mode	13
3.4.1	Goal	13
3.4.2	Inputs	13
3.4.3	Setup	13
3.4.4	Running STIEditor	13
3.4.5	Results	14
3.5	Import Pins Mode	14
3.5.1	Goal	14
3.5.2	Inputs	15
3.5.3	Running STIEditor	16
3.5.4	Results	16
4	Using STIEditor	17
4.1	Generating Setup File Template	17
4.2	STIEditor Commands	17
4.2.1	Signal Manipulation	18
4.2.2	Signal Groups Manipulation	19
4.2.3	Timing Manipulation	20
4.2.4	Vector Manipulation	21

4.2.5	State Manipulation	23
4.2.6	Processing Failures	27
4.2.7	Splitting and Joining Cycles	29
4.2.8	Statements Insertion	30
4.2.9	Compress Pattern	32
4.2.10	Merge Files	33
4.2.11	Merge Bursts	36
4.2.12	Split Pattern To Ports	37
4.2.13	Other Commands	37
4.3	Multiple Commands Spreadsheet File	39
5	Using STIEditor in Import Pins Mode	41
5.1	Example of import_pins Usage for a Single File	41
5.2	Example of import_pins Usage for Multiple Files	42
6	Appendix	43
6.0.1	STIEditor Global Configuration	43
6.0.2	STIL Writer Customization	43

Chapter 1

Introduction

1.1 Audience

STILEditor User Manual is intended for test and design engineers of integrated circuits.

1.2 Terms Used in This Manual

STIL *Standard Test Interface Language*, developed by IEEE for the design-to-test flow of the semiconductor test industry.

IEEE *Institute of Electrical and Electronics Engineers, Inc.*, continues to develop sophisticated yet complex modules to *STIL*.

1.3 Concept

STILEditor is designed to perform a set of operations on STIL files, namely importing pins usage information; sequential and parallel merge; edit operations including adding virtual signals, signal masking and state remapping, removal of vectors, extraction of vectors, compression (using loops) and many others. Every run of STILEditor is restricted to a specific operation mode (edit, merge or import_pins).

Chapter 2

Using STILEditor

STILEditor operates via a command line interface (CLI). It accepts a mandatory argument **mode**, and the **setup** argument, which is only mandatory when using **edit** and **merge** modes.

mode argument selects the required mode of operation. There are three modes: **edit**, **merge** and **import_pins**.

The **setup** argument specifies setup file. It is used to describe actions/flags applied to all input STIL files; these are defined using Python language format.

2.1 Synopsis

```
stileditor -mode <str> [-help] [-version] [-timestamp] [-licwait] [-template] [-setup <str>]
[-logfile <str>] [-workdir <str>] [STIL input file(s)...]
```

2.2 Command Line Options Description

The following are available command line options of STILEditor:

-mode	Select Working mode (edit merge import_pins).
-help	Display help message and exit.
-version	Show version number and exit.
-timestamp	Enable log timestamp.
-licwait	Wait for the license, if not available. The application will wait indefinitely until a license becomes available.
-template	Generate setup template and exit. A file containing all supported setup options will be generated.
-setup	Path to the setup file.
-logfile	Redirect log to the specified file.
-workdir	Path to the output directory.

Chapter 3

Usage Tutorial

3.1 Tutorial Goals

The goal of this tutorial is to familiarize user with STIEditor tool. After completing the tutorial you will be able to edit and manipulate STIL files. The tutorial is based on simple examples, located in the **\$ATE2STILHOME/labs/stileditor** directory. The tutorial assumes Linux operating system environment.

3.2 Editing Tutorial

3.2.1 Goals

The goal is to perform the following operations on the STIL file:

- Add virtual signals (fixed and copy) according to external datasheet file (virtual_signals.xls).
- Add loop at cycles 2 and 3 of the first pattern. Note that the original pattern contains a loop at the beginning. This existing loop will be unrolled to allow compression.
- Add a single vector loop on the last vector of the first pattern.
- Add Free Running Clock.
- Compress repeating vector groups in the second pattern into loops to reduce pattern size.

3.2.2 Inputs

The input file is a STIL file located in the **\$ATE2STILHOME/labs/stileditor/edit** directory. The file contains the following signals:

- clk : Clock signal.
- enable : Input signal.
- IN[0..9] : 10 Input data signals.
- data : Inout signal.
- out[0..1]: 2 Output data signal.

STIL Sample 3.1: Edit Input Pattern 1

```

1 Pattern "P1" {
2     W "t1";
3     // initialization sequence
4     V { "__ALL__"=PS0000000000BHH; }
5     Loop 3 {
6         V { "__ALL__"=PS1010000000BHH; }
7         V { "__ALL__"=PS0000000000BHH; }
8         V { "__ALL__"=PS0101000000BHH; }
9     }
10    // end initialization sequence
11    V { "__ALL__"=PS0000000000BHH; }
12    W "t2";
13    V { "__ALL__"=PS1111111111BLL; }
14    V { "__ALL__"=PS1111111111BLL; }
15    V { "__ALL__"=PS1111111111BLL; }
16    Loop 30 {
17        V { "__ALL__"=PS1111111111BLL; }
18    }
19    V { "__ALL__"=PR1111111111BXX; }
20    W "t3";
21    V { "__ALL__"=PS1111111111BLH; }
22    V { "__ALL__"=PS0000001111BLH; }
23    V { "__ALL__"=PS0000001111BLH; }
24    V { "__ALL__"=PS0000001111BLH; }
25    V { "__ALL__"=PS0000001111BLH; }
26    V { "__ALL__"=PS0000001111BLH; }
27    V { "__ALL__"=PS0000001111BLH; }
28    V { "__ALL__"=PS0011111111BHH; }
29    V { "__ALL__"=PS0011111111BHH; }
30 }

```

STIL Sample 3.2: Edit Input Pattern 2

```

1 Pattern "P2" {
2     W "t1";
3     V { "__ALL__"=PS0011111111BHH; }
4     V { "__ALL__"=PS1111111111BLL; }
5     V { "__ALL__"=PS0000001111BLH; }
6     V { "__ALL__"=PS0011111111BHH; }
7     V { "__ALL__"=PS1111111111BLL; }
8     V { "__ALL__"=PS0000001111BLH; }
9     V { "__ALL__"=PS0011111111BHH; }
10    V { "__ALL__"=PS1111111111BLL; }
11    V { "__ALL__"=PS0000001111BLH; }
12    V { "__ALL__"=PS0011111111BHH; }
13    V { "__ALL__"=PS1111111111BLL; }
14    V { "__ALL__"=PR1111111111BHL; }
15    V { "__ALL__"=PS0000000000BHH; }
16    V { "__ALL__"=PS0000001111BLH; }
17    V { "__ALL__"=PS1111111111BLL; }
18    V { "__ALL__"=PS0000000000BHH; }
19    V { "__ALL__"=PS0000001111BLH; }
20    V { "__ALL__"=PS1111111111BLL; }
21    V { "__ALL__"=PS0000000000BHH; }
22    V { "__ALL__"=PS0000001111BLH; }
23    V { "__ALL__"=PS1111111111BLL; }
24    V { "__ALL__"=PS0000000000BHH; }

```

```
25 V { " __ALL__ "=PS0000011111BLH; }
26 V { " __ALL__ "=PS1111111111BLL; }
27 }
```

3.2.3 Setup

In the setup example there are five unique commands:

- 'add_loop' to add loop and repeat structure.
- 'add_signals_copy_file' to add virtual signal by copy.
- 'add_signals_fixed_file' to add fixed virtual signal.
- 'add_free_run_clk' to add free running clock.
- 'compress' to decrease pattern size by compressing existing vectors.

For simplicity, all the commands are executed on input pattern P1, except for the 'compress' command which is executed on pattern P2.

In addition to the setup file, there is an XLSX file containing data for 'add_signals_copy_file' and 'add_signals_fixed_file' commands. Both commands specify path to that file, but with different sheet names. The 'add_signals_copy_file' command would get its data from the 'copy_virtual' sheet and the 'add_signals_fixed_file' command would get its data from the 'fixed_virtual' sheet.

The first 'add_loop' command specifies the 'cycle_start' parameter but does not specify the 'cycle_end' parameter, meaning that the command is running on one cycle, so a single vector loop is created.

The second 'add_loop' command specifies both 'cycle_start' and 'cycle_end' parameters (2 and 3 respectively) It means that the created loop will contain 2 vectors.

The 'add_free_run_clk' would add free running clock named 'clk2' to the output STIL file.

The 'compress' command specifies the limits of the generated loops, such that there will be a minimum of 2 vectors in the loop body, maximum of 50 vectors in the loop body, there won't be loops with less than 2 repetitions, no maximum limit of repetitions, and the total number of cycles in each generated loops will be at least 4. Furthermore, scans and loops in the input won't be expanded, and comments in the pattern won't break compressing sequences (they will have no affect on the loops).

3.2.4 Running STIEditor

In order to run the program, set the setup file to 'setup.py'.

```
command line
$ ATE2STILHOME/bin/stileditor -setup setup.py
```

3.2.5 Results

The demonstration shows the following:

- We can add virtual signals (both fixed signals and copies of existing signals).

- Free Running Clock is added to the signal groups according to the specified parameters.

STIL Sample 3.3: Edit Output Signals

```

1 UserKeywords FreeRunning;
2
3 Signals {
4     "clk" In;
5     "enable" In;
6     "IN[0]" In;
7     "IN[1]" In;
8     .
9     .
10    .
11    "IN[8]" In;
12    "IN[9]" In;
13    "data" InOut;
14    "out[0]" Out;
15    "out[1]" Out;
16    "enable_cp" In;
17    "data_cp" InOut;
18    "out0_cp" Out;
19    "trigger_measure" In;
20    "trigger_capture" In;
21    "clk2" InOut;
22 }
23 SignalGroups {
24     "__ALL__" = '"clk"+"enable"+"IN[0]" + ... + "IN[8]" + "IN[9]"
25     + "data" + "out[0]" + "out[1]"';
26     "clk2_refclk" = '"clk2"' { FreeRunning { Period '500ns';
27     LeadingEdge '100ns'; TrailingEdge '400ns'; OffState X;
28     } }
29     "__ALL__1" = '"clk"+"enable"+"IN[0]" + "IN[1]" ... + "IN[8]"
30     + "IN[9]" + "data" + "out[0]" + "out[1]" + "enable_cp" + "data_cp"
31     + "out0_cp" + "trigger_measure" + "trigger_capture"';
32 }

```

- We can add loops anywhere in the pattern, including cycles that are already located in existing loop.

STIL Sample 3.4: Edit Output Pattern 1

```

1 Pattern "P1" {
2     W "t1";
3     // initialization sequence
4     V { "__ALL__1" = PS0000000000BHHSBH01; }
5     Loop 10
6         V { "__ALL__1" = PS1010000000BHHSBH01; }
7         V { "__ALL__1" = PS0000000000BHHSBH01; }
8     }
9     V { "__ALL__1" = PS0101000000BHHSBH01; }
10    V { "__ALL__1" = PS1010000000BHHSBH01; }
11    V { "__ALL__1" = PS0000000000BHHSBH01; }
12    V { "__ALL__1" = PS0101000000BHHSBH01; }
13    V { "__ALL__1" = PS1010000000BHHSBH01; }
14    V { "__ALL__1" = PS0000000000BHHSBH01; }
15    V { "__ALL__1" = PS0101000000BHHSBH01; }
16    // end initialization sequence

```

```
17 V { "__ALL__1"=PS0000000000BHHSBH01; }
18 W "t2";
19 V { "__ALL__1"=PS1111111111BLLSBL01; }
20 V { "__ALL__1"=PS1111111111BLLSBL01; }
21 V { "__ALL__1"=PS1111111111BLLSBL01; }
22 Loop 30 {
23     V { "__ALL__1"=PS1111111111BLLSBL01; }
24 }
25 V { "__ALL__1"=PR1111111111BXXRBX01; }
26 W "t3";
27 V { "__ALL__1"=PS1111111111BLHSBL01; }
28 V { "__ALL__1"=PS0000011111BLHSBL01; }
29 V { "__ALL__1"=PS0000011111BLHSBL01; }
30 V { "__ALL__1"=PS0000011111BLHSBL01; }
31 V { "__ALL__1"=PS0000011111BLHSBL01; }
32 V { "__ALL__1"=PS0000011111BLHSBL01; }
33 V { "__ALL__1"=PS0000011111BLHSBL01; }
34 V { "__ALL__1"=PS0011111111BHHSBH01; }
35 Loop 20 {
36     V { "__ALL__1"=PS0011111111BHHSBH01; }
37 }
38 }
```

- The compression algorithm reduced the number of vectors in the pattern from 24 to 9, without changing the pattern's functionality.

STIL Sample 3.5: Edit Output Pattern 1

```
1
2 Pattern "P2" {
3     W "t1";
4     Loop 3 {
5         V { "__ALL__1"=PS0011111111BHHSBH01; }
6         V { "__ALL__1"=PS1111111111BLLSBL01; }
7         V { "__ALL__1"=PS0000011111BLHSBL01; }
8     }
9     V { "__ALL__1"=PS0011111111BHHSBH01; }
10    V { "__ALL__1"=PS1111111111BLLSBL01; }
11    V { "__ALL__1"=PR1111111111BHLSBL01; }
12    Loop 4 {
13        V { "__ALL__1"=PS0000000000BHHSBH01; }
14        V { "__ALL__1"=PS0000011111BLHSBL01; }
15        V { "__ALL__1"=PS1111111111BLLSBL01; }
16    }
17 }
```

3.3 Pattern Merge - Serial Mode

3.3.1 Goal

Merge between two STIL files, creating an output file with combined timing and vectors. Input files contain three patterns altogether. In this case STIL Editor is configured to merge patterns serially, that is the resulting pattern will contain vectors from the first pattern, then vectors from the second etc.

3.3.2 Inputs

The inputs are two STIL files located in the **\$ATE2STILHOME/labs/stileditor/merge_serial** directory. Each pattern uses different periods to demonstrate WaveformTable split.

Each pattern contains 10 signals:

- clk : Clock signal
- enable : Input signal
- IN[0..3] : 4 Inputs data signals.
- data : Inout signal
- out[0..2]: 3 Outputs data signal.

3.3.3 Setup

In this setup example there is only one command:

```
1 input_files=['simple_P1_P2.stil']
2 merge_files(
3     files = ['simple_P3.stil'],
4     merge_patterns=1,
5     unify_periods=1,
6     merge_mode='serial',
7     use_file_name_as_pat=0,
8 )
```

The 'merge_patterns' parameter is set to 1, this means STIL Patterns are merged into a single Pattern; other option is keeping separate patterns but combining them using PatternBurst. The 'merge_mode' parameter specifies that STIL patterns are merged in a 'serial' way as explained above. The last parameter 'unify_periods' is set to 1, it forces STILEditor to generate single WaveformTable. Note that inputs contain multiple WaveformTables with different periods. Although it does not prevent merging patterns serially, in this case STILEditor is configured to create a single unified WaveformTable with period which is a GCD of the input periods; pattern vectors are split accordingly.

3.3.4 Running STILEditor

In order to run the program, set the setup file to 'setup.py'.

```
_____ command line _____
$ ATE2STILHOME/bin/stileditor -setup setup.py
```

3.3.5 Results

The output is STIL file with cumulative timing and vectors. The PatterBurst section includes concatenated patterns in a single Pattern object. The output pattern, shown below, contains vectors from all input patterns. Original pattern names are used as pattern labels to indicate the start of each pattern.

STIL Sample 3.6: Serial Merge Output Pattern

```
1 Pattern "master" {
2     // File: simple_P1_P2.stil
3     // Pattern: P1
4     "P1":
```

```
5 W "WFT";
6 Loop 20 {
7     V { "__ALL__"=PS0000BHHH; }
8     V { "__ALL__"=PS0000BHHH; }
9 }
10 // File: simple_P1_P2.stil
11 // Pattern: P2
12 "P2":
13 Loop 15 {
14     V { "__ALL__"=pS11110LLL; }
15     V { "__ALL__"=pS11110LLL; }
16 }
17 // File: simple_P3.stil
18 // Pattern: P3
19 "P3":
20 Loop 20 {
21     V { "__ALL__"=PS00111LHH; }
22     V { "__ALL__"=US0011XXXX; }
23 }
24 }
```

3.4 Pattern Merge - Parallel Mode

3.4.1 Goal

Merge two patterns in parallel mode, as if the patterns were run simultaneously. As in the serial merge example, inputs contain multiple WaveformTables with different periods. But unlike serial mode, in parallel mode STILEditor always creates a single unified WaveformTable.

3.4.2 Inputs

The inputs are two STIL files located in the **\$ATE2STILHOME/labs/stileditor/merge_parallel** directory. Each of them contains 6 signals (1 clock, 4 data, 1 output) and one pattern. Note that the STIL files contain different signals in order to allow parallel merge. Each pattern uses different period to demonstrate WaveformTable unification.

3.4.3 Setup

In the setup example there is only one command:

```
1 input_files=['input_1.stil']
2 merge_files(
3     files = ['input_2.stil'],
4     merge_patterns=1,
5     merge_mode='parallel',
6 )
```

'merge_patterns' is set to 1, this means STIL Patterns are merged into a single pattern. 'merge_mode' specifies that STIL Patterns are merged into a 'parallel' pattern.

3.4.4 Running STILEditor

In order to run the program, set the setup file to 'setup.py'

```
command line
$ ATE2STILHOME/bin/stileditor -setup setup.py
```

3.4.5 Results

In the output, one pattern has been created. It contains a merged list of signals and cumulative timing. The output pattern, shown below, contains data from both input patterns.

STIL Sample 3.7: Parallel Merge Output Pattern

```

1
2 SignalGroups {
3   "__ALL__" = '"clk1"+"data[0]"+"data[1]"+"data[2]"+"data[3]"+"
   "out1"+"clk2"+"data[4]"+"data[5]"+"data[6]"+"data[7]"+"out2"
   '
4 }
5
6 Pattern "master" {
7   W "WFT";
8   V { "__ALL__"=P0000LpUUUUH; }
9   V { "__ALL__"=P1111LDUUUUH; }
10  V { "__ALL__"=P0000LpDDDDH; }
11  V { "__ALL__"=P1111LDDDDX; }
12  V { "__ALL__"=P0000LpUUUUH; }
13  V { "__ALL__"=P1111LDUUUUH; }
14  .
15  .
16  .
17  V { "__ALL__"=U1111XDUUUH; }
18  V { "__ALL__"=U1111XpDDDDH; }
19  V { "__ALL__"=U1111XDDDDX; }
20  V { "__ALL__"=U1111XpUUUUH; }
21  V { "__ALL__"=U1111XDUUUH; }
22  V { "__ALL__"=U1111XpDDDDH; }
23  V { "__ALL__"=U1111XDDDDX; }
24 }
```

3.5 Import Pins Mode

3.5.1 Goal

Combine data from multiple STIL inputs, in order to provide the user with pin usage information for all the patterns included in the source files. The pin usage information is generated in a comma-separated file (CSV).

Each unique pattern name will have a matching column, and each unique signal name will have a matching row. The first row is reserved for the pin name, the second row is used for the pin type, and the rest of the rows are used for the pattern names. If a pin appears at least once in a pattern, the cell matching the row of that pin and column of the containing pattern will have the value **Y**, and will otherwise have the value **N**. The inputs contain multiple patterns and signals, some of which may have identical names, and therefore will be represented in the output by the same columns and rows, respectively.

3.5.2 Inputs

The inputs are two STIL files located in the **\$ATE2STILHOME/labs/stileditor/import_pins** directory. Each file contains two pattern blocks and four pins: one clock, two data pins, one output and one supply pin. The clock, output and supply pins in both files share the same name, while all the other pins have unique names. Each pattern uses exactly one clock pin, one data pin, and one output in.

STIL Sample 3.8: Import Pins Input1

```
1
2 Signals {
3     "clk" In;
4     "Input1_data1" In;
5     "Input1_data2" In;
6     "out" Out;
7     "sup" Supply;
8 }
9
10 Pattern "Input1_Pat1" {
11     W "t1";
12     V { "clk"=P; "Input1_data1"=0; }
13     V { "clk"=P; "Input1_data1"=0; "out"=H; }
14     V { "clk"=P; "Input1_data1"=1; "out"=H; }
15 }
16
17 Pattern "Input1_Pat2" {
18     W "t1";
19     V { "clk"=P; "Input1_data2"=1; }
20     V { "clk"=P; "Input1_data2"=1; "out"=L; }
21     V { "clk"=P; "Input1_data2"=0; "out"=H; }
22     V { "clk"=P; "Input1_data2"=1; "out"=L; }
23 }
```

STIL Sample 3.9: Import Pins Input2

```
1
2 Signals {
3     "clk" In;
4     "Input2_data1" In;
5     "Input2_data2" In;
6     "out" Out;
7     "sup" Supply;
8 }
9
10 Pattern "Input2_Pat1" {
11     W "t1";
12     V { "clk"=P; "Input2_data1"=0; }
13     V { "clk"=P; "Input2_data1"=1; "out"=L; }
14     V { "clk"=P; "Input2_data1"=0; "out"=H; }
15 }
16
17 Pattern "Input2_Pat2" {
18     W "t1";
19     V { "clk"=P; "Input2_data2"=1; }
20     V { "clk"=P; "Input2_data2"=0; "out"=L; }
21     V { "clk"=P; "Input2_data2"=0; "out"=L; }
22 }
```

3.5.3 Running STIEditor

In order to run the program, use switch 'import_pins' and specify input files 'Input1.stil' and 'Input2.stil'

```
command line
$ ATE2STILHOME/bin/stileditor -import_pins Input1.stil Input2.stil
```

3.5.4 Results

The file *imported.csv* has been created in the output directory. It contains a combined table showing which signals were used in the patterns, as seen in the table below.

Table 3.1: Import Pins Output Table

# Pin Name	Pin Type	Active [Input1_Pat1]	Active [Input1_Pat2]	Active [Input2_Pat1]	Active [Input2_Pat2]
clk	In	Y	Y	Y	Y
Input1_data1	In	Y	N	N	N
Input1_data2	In	N	Y	N	N
out	Out	Y	Y	Y	Y
sup	Supply	N	N	N	N
Input2_data1	In	N	N	Y	N
Input2_data2	In	N	N	N	Y

Chapter 4

Using STILEditor

STILEditor is designed to create a STIL file by editing and manipulating an existing one. The behavior of STILEditor can be customized via commands in the setup file.

4.1 Generating Setup File Template

The following command generates 2 files:

- A setup file template containing a complete list of all edit configurations and commands.
- A template XLSX file for multiple signal manipulation commands (see [4.3](#))

```
> stileditor -template
```

4.2 STILEditor Commands

The setup file contains global configurations (see [6.0.1](#)), STIL writer customization (see [6.0.2](#)) settings and edit commands.

Notes:

- Each command is applied on the results of the previous command, not on the original file.
- In commands that require to specify cycle number, the cycle count starts at 1.
- The reserved words 'pat_start' and 'pat_end' indicate the start and the end of the pattern respectively.
- 'allow_expand_loops' parameter - controls whether loops could be expanded for command execution.
STILEditor would exit with an error if expand loop is required but not allowed.
- 'pat_name' parameter - Selects which pattern will be affected by the command, when possible. To apply the command to all the available patterns, use an empty string as a value.

4.2.1 Signal Manipulation

There are five types of operations on signals:

- add signal
- add fixed signal
- add copy signal
- delete signal
- rename signal

To perform multiple signal manipulations commands, the user may arrange the commands in a tabular file (see 4.3). A spreadsheet for multiple signal manipulations can also be created by generating a setup file template (see 4.1).

Add Signal

'*add_signal*' command adds signal to the Signals block.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	New signal name.
sig_type	Yes	In/Out/InOut	New signal type.

Example

```
add_signal(
    name      = 'new_sig',
    sig_type = 'In',
)
```

Add Fixed Signal

'*add_signal_fixed*' command adds virtual signal with fixed state.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	New signal name.
sig_type	No	In/Out/InOut	New signal type. If omitted, derived from state.
state	Yes	D/U/Z/L/H/X/T	Signal fixed state.
time	No	<i>string</i>	Event time. Default is '0ns'.
pat_name	No	<i>string</i>	Pattern name. If omitted, signal is added to all patterns (serial burst) or to the longest pattern (parallel burst).

Example

```
add_signal_fixed(
    name      = 'new_fix_sig',
    sig_type = 'InOut',
    state     = 'U',
    time      = '5ns'
)
```

Add Copy Signal

'*add_signal_copy*' command adds virtual signal which is a copy of an existing signal.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	New signal name.
sig_type	No	In/Out/InOut	New signal type. Default is the type of the reference signal.
ref_name	Yes	<i>string</i>	Name of the signal to copy from.
invert	No	<i>boolean</i>	Invert state flag. Default is 0. If set to 1, the command inverts original signal values.
pat_name	No	<i>string</i>	Pattern name. If omitted, signal is added to all patterns (serial burst) or to the longest pattern (parallel burst).

Example

```
add_signal_copy(  
    name      = 'new_sig_name',  
    ref_name  = 'invert_sig',  
    invert    = 1  
)
```

Delete Signal

'*delete_signal*' command deletes STIL signal.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	Name of the signal to delete.

Example

```
delete_signal(  
    name = 'sig2'  
)
```

Rename Signal

'*rename_signal*' command renames STIL signal.

Parameter	Req'd	Value	Description
from_name	Yes	<i>string</i>	Original signal name.
to_name	Yes	<i>string</i>	New signal name.

Example

```
rename_signal(  
    from_name = 'sig_name',  
    to_name   = 'new_sig_name'  
)
```

4.2.2 Signal Groups Manipulation

Add Signal

'*add_signal_group*' command adds group to SignalGroups block.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	New group name.
members	No	<i>list</i>	New group members (signals or groups). Default is empty list.

Example

```
add_signal_group(
    name    = 'new_group',
    members = ['S1', 'S2'],
)
```

4.2.3 Timing Manipulation

Add Waveform

'add_waveform' command adds STIL waveform to signal(s). Use 'name' parameter to select signal(s) for which the new timing will be added, by providing signal name, signal group name or signal type.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	Signal name, signal group name or type. Supported types are In, Out, InOut.
wft	No	<i>string</i>	WFT name. If empty, add waveform to all WFTs.
wfc	Yes	<i>char</i>	WFC of the new waveform.
waveform	Yes	<i>string</i>	Definition of events of the new waveform. Format is 'time1:action1; time2:action2; ...', using valid STIL time expressions and actions. Valid STIL actions are: DUNPZLHTVXIhtvx-ABF?RGQM.
override	No	<i>boolean</i>	If 1, override existing waveform. Default is 0.

Example

```
add_waveform(
    name    = 'sig_name',
    wft     = 'wft_name',
    wfc     = 'P',
    waveform = '0ns: D; td:U', # td is a spec variable
    override = 0
)
```

Remap Signal Actions

'remap_signal_actions' command replaces event actions in STIL timing. Use 'name' parameter to select signal(s) for which actions will be replaced, by providing signal name, signal group name or signal type.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	Signal name, signal group name or type. Supported types are In, Out, InOut.

from_action	Yes	string	Old action. Supported values are either one or more valid STIL action(s) DUNPZLHTVXI-htvxABF?RGQM or * wild card.
to_action	Yes	char	New action. Single valid STIL action.

Example

```
remap_signal_actions(  
    name      = 'sig_name',  
    from_action = 'LH',  
    to_action  = 'X'  
)
```

Clear Unused Timing

'clear_unused_timing' command removes unused timing from STIL.

Parameter	Req'd	Value	Description
clear_wfts	No	boolean	Remove unused WFTs. Default is 1.
clear_wavs	No	boolean	Remove unused waveforms. Default is 1.
clear_mixed_wavs	No	boolean	Remove unused mixed inout waveforms. If set to 1 and clear_wavs is 0, the tool will only remove waveforms which contain drive on event eventually followed by drive off event. Default is 0.

Example

```
clear_unused_timing(  
    clear_wfts = 1,  
    clear_wavs = 0,  
    clear_mixed_wavs = 1  
)
```

Force Edge Strobe

'force_edge_strobe' command replaces window strobe events with edge strobe events.

Example

```
force_edge_strobe()
```

4.2.4 Vector Manipulation

Delete Vectors

'delete_vectors' command deletes the vectors within the defined cycles range. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	string	Pattern name. If empty, all patterns are processed.

cycle_start	Yes	<i>integer</i>	Range start. Supported values are pat_start, pat_end or positive integer cycle number.
cycle_end	No	<i>integer</i>	Range end. Supported values are pat_start, pat_end or positive integer cycle number. Default is the same as cycle_start.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```
delete_vectors(
    pat_name      = 'Pattern1',
    cycle_start   = pat_start,
    cycle_end     = 12,
    allow_expand_loops = 0
)
```

Extract Vectors

'*extract_vectors*' command leaves only the vectors in the defined cycles range. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are selected. Non-selected patterns are removed.
cycle_start	Yes	<i>integer</i>	Range start. Supported values are pat_start, pat_end or positive integer cycle number.
cycle_end	Yes	<i>integer</i>	Range end. Supported values are pat_start, pat_end or positive integer cycle number.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```
extract_vectors(
    pat_name      = 'Pattern2',
    cycle_start   = 3,
    cycle_end     = 50,
    allow_expand_loops = 0
)
```

Extract Vectors By Labels

'*extract_vectors_by_labels*' command leaves only the vectors located between the specified labels. Note that vectors located before the first scan statement are considered pattern setup vectors. These vectors can be either kept or removed, depending on the value of the 'keep_setup_vectors' parameter.

Parameter	Req'd	Value	Description
-----------	-------	-------	-------------

pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are selected. Non-selected patterns are removed.
label_start	Yes	<i>string</i>	Range start.
label_end	Yes	<i>string</i>	Range end.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.
keep_setup_vectors	No	<i>boolean</i>	Keep pattern setup vectors. Default is 1.

Example

```
extract_vectors(  
    pat_name      = 'Pattern2',  
    label_start   = 'Lab1',  
    label_end     = 'Lab2',  
    allow_expand_loops = 0,  
    keep_setup_vectors = 1,  
)
```

Insert Vectors

'insert_vectors' command adds vectors from another STIL file to a specific cycle in our pattern. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
cycle	Yes	<i>integer</i>	Insertion cycle number. Inserted vectors will be added after the specified cycle. Supported values are pat_start, pat_end or positive integer cycle number.
path	Yes	<i>string</i>	Path to the STIL file to take inserted vectors from.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```
insert_vectors(  
    pat_name      = '',  
    cycle         = 7,  
    path          = 'vectors.stil',  
    allow_expand_loops = 0,  
)
```

4.2.5 State Manipulation

Set Signal State

'set_signal_state' command allows setting pattern state for signal which is not used in that pattern in the input file.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	Signal name.

state	Yes	<i>char</i>	WFC state.
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.

Example

```
set_signal_state(
    name      = 'data',
    wfc       = '0',
    pat_name  = 'P1'
)
```

Mask Signal States

'*mask_signal_states*' command allows masking by identifying sequences within a specific cycle range. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
name	Yes	<i>string</i>	Signal name, signal group name or type. Supported types are In, Out, In-Out.
sequence	Yes	<i>string</i>	Actions sequence to mask.
cycle_start	No	<i>integer</i>	Range start. Supported values are pat_start, pat_end or positive integer cycle number. Default is pat_start.
cycle_end	No	<i>integer</i>	Range end. Supported values are pat_start, pat_end or positive integer cycle number. Default is pat_end.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```
mask_signal_states(
    pat_name      = 'Pattern1',
    name          = 'sig1_name',
    sequence      = 'XH',
    cycle_start   = pat_start,
    cycle_end     = 50,
    allow_expand_loops = 1
)
```

Replace Signal States

'*replace_signal_states*' command allows replacing one or more waveform characters specified by from_state value to a new value specified by 'to_state' waveform character. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.

name	Yes	<i>string</i>	Signal name, signal group name or type. Supported types are In, Out, In-Out.
from_state	Yes	<i>string</i>	Original WFC state list or * wildcard.
to_state	Yes	<i>char</i>	Target WFC state.
cycle_start	No	<i>integer</i>	Range start. Supported values are pat_start, pat_end or positive integer cycle number. Default is pat_start.
cycle_end	No	<i>integer</i>	Range end. Supported values are pat_start, pat_end or positive integer cycle number. Default is pat_end.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```
replace_signal_states(  
    pat_name      = 'Pattern2',  
    name          = 'sig2_name',  
    from_state    = '01K',  
    to_state      = 'P',  
    cycle_start   = 122,  
    cycle_end     = pat_end,  
    allow_expand_loops = 0  
)
```

Apply Sequence Map

'*apply_sequence_map*' command uses map of state sequences to modify pattern vectors data.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
name	Yes	<i>string</i>	Signal name, signal group name or type. Supported types are In, Out, InOut.
sequence_map	Yes	<i>dictionary</i>	Sequence map.
cycle_start	No	<i>integer</i>	Range start. Supported values are pat_start, pat_end or positive integer cycle number. Default is pat_start.
cycle_end	No	<i>integer</i>	Range end. Supported values are pat_start, pat_end or positive integer cycle number. Default is pat_end.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```

apply_sequence_map(
    pat_name      = 'P1',
    name          = 'data',
    sequence_map  = 'LHL': 'LXL', 'LLL': 'LXL' ,
    cycle_start   = pat_start,
    cycle_end     = pat_end,
    allow_expand_loops = 0
)

```

Sequence map consists of pairs of 'from sequence' and 'to sequence'. States used both sequences are no WFCs, but rather drive or compare actions used in the relevant waveforms. Actions that can be used in sequence map are D, U, Z, L, H, T, X. Referencing waveforms from sequence map should follow the below rules.

- Waveforms containing actions other than DUPZLHTXlhtx are not supported, i.e. such waveform cannot be referenced from sequence map.
- Empty waveforms are not supported.
- Waveforms with multiple different drive/compare actions (e.g. {DUD}) are not supported.
- Duplicated actions are ignored, e.g. {LL} waveform should be referenced as L.
- P action is ignored, e.g. {PD} should be referenced as D.
- X and Z action are ignored if there is different drive/compare, e.g. {ZL} should be referenced as L, but {Z} should be referenced as Z.
- If both Z and X and only them are present (e.g. {ZX}), the waveform should be referenced as X.
- Window compare actions are represented by matching edge compare actions, e.g. {lx} should be referenced as L.

Lets assume that input file contains the following timing and pattern.

```

Example
Timing {
1  WaveformTable t1 {
2      Period '100ns';
3      Waveforms {
4          data { 012 { '0ns' Z; '80ns' l/h/x; '90ns' x; } }
5      }
6  }
7  }
8  }
9  ...
10 Pattern P1 {
11     W t1;
12     V { data = 2; }
13     V { data = 0; }
14     V { data = 1; }
15     V { data = 0; }
16     V { data = 2; }
17 }

```

Pattern states for signal 'data' are 20102, meaning actions XLHLX. Using the setup described above replaces LHL with LXL, that is WFC 1 in cycle 3 is replaced by WFC 2.

Example

```
1 Pattern P1 {  
2   W t1;  
3   V { data = 2; }  
4   V { data = 0; }  
5   V { data = 2; }  
6   V { data = 0; }  
7   V { data = 2; }  
8 }
```

Force Pattern WFT

'force_wft' command changes WFT used in the specified pattern cycle(s).

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
wft_name	Yes	<i>string</i>	Forced WFT name.
cycle_start	Yes	<i>integer</i>	Range start. Supported values are pat_start, pat_end or positive integer cycle number.
cycle_end	No	<i>integer</i>	Range end. Supported values are pat_start, pat_end or positive integer cycle number. Default is same as cycle_start.
allow_expand_loops	No	<i>boolean</i>	Expand loops if required. Default is 0.

Example

```
force_wft(  
  pat_name      = 'P1',  
  wft_name      = 't2',  
  cycle_start   = 20,  
  cycle_end     = pat_end,  
  allow_expand_loops = 0  
)
```

4.2.6 Processing Failures

'mask_failures' command causes STIEditor to read failures information file and to use information from this file to mask waveforms for specified signals in specified cycles. Masked waveforms are replaced in pattern by waveform with Z and X actions. If such waveform exists, it is reused. Otherwise, new waveform is created.

'fix_failures' command acts in a similar way, but replaces failed compare events with actual events.

If failure occurs inside loop, the loop will be split to three parts: iterations before the failure, iteration with the failure, iterations after the failure. If there are more failures in subsequent loop iterations, they will be handled in the same way.

In scan vectors, mask/fix failures are allowed only in scan signals. Scan vectors are not expanded.

Parameter	Req'd	Value	Description
fail_file	Yes	<i>string</i>	Path to failures file.
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.

Example

```
mask_failures(
    fail_file = 'failures.txt',
    pat_name = 'P1',
)
```

Failures information file is tab-separated text file with four columns:

- cycle number
- signal name
- expected data - compare value (0 for low, 1 for high) expected in STIL pattern
- actual data - compare value which actually occurred on ATE

For example, the below file means that in cycle 2 ATE expected high, but received low; in cycle 3 the opposite occurred.

Example

```
2 data 1 0
3 data 0 1
```

Lets assume we have input STIL with the following timing and pattern:

Example

```
1 Timing {
2     WaveformTable t1 {
3         Period '100ns';
4         Waveforms {
5             data { LH { '0ns' Z; '80ns' L/H; } }
6         }
7     }
8 }
9 ...
10 Pattern P1 {
11     W t1;
12     V { data = L; }
13     V { data = H; }
14     V { data = L; }
15 }
```

Using setup and failures file described above results in the following output. Note that new mask waveform with WFC X is created.

Example

```
1 Timing {
2     WaveformTable "t1" {
3         Period '100ns';
4         Waveforms {
5             "data" {
6                 LH { '0ns' Z; '80ns' L/H; }
7                 X { '0ns' Z; '0ns' X; }
8             }
9         }
10    }
11 }
12 ...
13 Pattern "P1" {
14     W "t1";
15     V { "data"=L; }
16     V { "data"=X; }
17     V { }
18 }
```

4.2.7 Splitting and Joining Cycles

Split WFT

'*split_wft*' command splits STIL cycles to the specified number of sub-cycles. The command has the following effect:

- Periods of the specified WaveformTables are divided by split count. All waveforms are split to subwaveforms, each containing events relevant to a certain sub-cycle. E.g. let's assume that WaveformTable period is 100ns and split count is 2. In this case, new period will be 50ns. Also, waveform with events at 0ns, 40ns and 60ns will be split to two waveforms - one with events of the first sub cycle, that is at 0ns and 40ns; and one with events of the second sub cycle, that is at 60ns - 50ns = 10ns.
- Pattern vectors which use the specified WaveformTables are split to sub vectors.

Parameter	Req'd	Value	Description
split_count	Yes	<i>integer</i>	Split count.
wfts	No	<i>list</i>	List of WaveformTables that should be split. By default the command is applied to all WaveformTables.
expand_scan	No	<i>boolean</i>	Expand scan flag. Default is 0. To replace scan vectors with regular vectors, set to 1.

Example

```
split_wft(
    split_count = 2,
    wfts        = ['WFT1','WFT2'],
    expand_scan  = 0
)
```

Unify WFT Period

'*unify_wft_period*' command has functionality similar to '*split_wft*'. However, it automatically calculates split count for each WaveformTable so that all timing in the resulting STIL will have only one period (GCD of the original periods).

Parameter	Req'd	Value	Description
min_period	No	<i>string</i>	Min allowed period. Default is min input period divided by 10000. If the command results in period smaller than the specified value, STIEditor stops with error.
global_wft	No	<i>boolean</i>	Global WFT flag. Default is 1. Set to 1 in order to place all sub-cycle waveforms into a single global WaveformTable. Otherwise each original WaveformTable is split to its own sub-tables.

Example

```
unify_wft_period(
    min_period = '10ns',
    global_wft = 1
)
```

Join Vector Blocks

'*join_vector_blocks*' command converts a block of sequential vectors to a single vector. This also necessitates combining WaveformTables of individual vectors into new WaveformTables for blocks.

Parameter	Req'd	Value	Description
block_length	Yes	<i>integer</i>	Number of vectors in a block.
max_wft_count	No	<i>integer</i>	Max allowed number of generated WFTs. Default is 0 meaning unlimited.

Example

```
join_vector_blocks(
    block_length = 2,
    max_wft_count = 0
)
```

4.2.8 Statements Insertion

Add User Keyword

'*add_user_keyword*' command inserts UserKeyword into a pattern at the specified cycle. Statements already occupying the specified cycle and the rest of the pattern will be shifted down. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
cycle	Yes	<i>integer</i>	Cycle number. Supported values are pat_start, pat_end or positive integer cycle number.
id	Yes	<i>string</i>	User keyword id.
text	No	<i>string</i>	User keyword text.
allow_expand_loops	No	<i>integer</i>	Expand loops flag. Default is 0.

Example

```
add_user_keyword(  
    pat_name      = 'Pattern2',  
    cycle         = pat_end,  
    id            = 'my_keyword',  
    text          = '{ my keyword block contents; }',  
    allow_expand_loops = 0  
)
```

Add IddqTestPoint

'*add_iddq*' command inserts IddqTestPoint statement into a pattern at the specified cycle. Statements already occupying the specified cycle and the rest of the pattern will be shifted down. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
cycle	Yes	<i>integer</i>	Cycle number. Supported values are pat_start, pat_end or positive integer cycle number.
allow_expand_loops	No	<i>integer</i>	Expand loops flag. Default is 0.

Example

```
add_iddq(  
    pat_name      = 'Pattern3',  
    cycle         = 21,  
    allow_expand_loops = 1  
)
```

Add Comment

'*add_comment*' command attaches comment to a vector at the specified cycle. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
cycle	Yes	<i>integer</i>	Cycle number. Supported values are pat_start, pat_end or positive integer cycle number.
text	Yes	<i>string</i>	Comment text.
comment_type	No	C / C++ / Annotation	Comment type.
allow_expand_loops	No	<i>integer</i>	Expand loops flag. Default is 0.

Example

```
add_comment(  
    pat_name      = 'Pattern1',  
    cycle         = 1,  
    text          = 'first cycle comment',  
    comment_type   = 'C++',  
    allow_expand_loops = 0  
)
```

Add Loop

'*add_loop*' command wraps a group of vectors within a loop. Note that initial cycle number is 1.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
loop_count	Yes	<i>integer</i>	Loop count.
cycle_start	Yes	<i>integer</i>	First loop vector cycle number. Supported values are pat_start, pat_end or positive integer cycle number.
cycle_end	No	<i>integer</i>	Last loop vector cycle number. Default is same as cycle_start. Supported values are pat_start, pat_end or positive integer cycle number.
allow_expand_loops	No	<i>integer</i>	Expand loops flag. Default is 0.

Example

```
add_loop(
    pat_name      = '',
    loop_count    = 4,
    cycle_start   = 3,
    cycle_end     = 10,
    allow_expand_loops = 0
)
```

4.2.9 Compress Pattern

The '*compress*' command is designed to minimize the total number of vectors in STIL Pattern blocks, by using of Loop statements for sequences of identical vectors. Both single and multivector loops are supported. The compression behavior can be customized by command configurations.

There are five parameters for controlling the boundaries of the loop structures:

'min_loop_size' and 'max_loop_size' to limit the loop's body size.

'min_loop_count' and 'max_loop_count' to limit the number of times the loop will repeat.

'min_itr_count' to prevent the creation of smaller loops (loop size * loop count).

If 'expand_loops' or 'expand_scan' is set to 1, STIEditor will expand all loops or scan statements accordingly, and then compression will be performed. Otherwise, STIEditor will treat complex structures (loop or scan) as a single pattern statement like vectors.

In order to ignore comments between vectors during compression, the flag 'ignore_comments' should be enabled. In this case, comments within a sequence of similar vector statement groups will not cut off the sequence during loop creation.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
min_loop_size	No	<i>integer</i>	Minimum loop size. Default is 2.

max_loop_size	No	<i>integer</i>	Maximum loop size. Default is 50.
min_loop_count	No	<i>integer</i>	Minimum loop counter. Default is 2.
max_loop_count	No	<i>integer</i>	Maximum loop counter. Default is -1 meaning unlimited.
min_itr_count	No	<i>integer</i>	Minimum iterations counter, i.e. loop size * loop count. Default is 4.
expand_loops	No	<i>boolean</i>	Expand loops flag. Default is 0.
expand_scan	No	<i>boolean</i>	Expand scan flag. Default is 0.
ignore_comments	No	<i>boolean</i>	Ignore comments flag. Default is 1.

Example

```
compress(  
  pat_name      = 'Pattern1',  
  min_loop_size = 2,  
  max_loop_size = 20,  
  min_loop_count = 2,  
  max_loop_count = 5,  
  min_itr_count = 10,  
  expand_loops  = 0,  
  expand_scan   = 0,  
  ignore_comments = 1  
)
```

For the example above, STIEditor will only create loops with a body that has between 2 and 20 statements, the number of repetitions will be between 2 and 5 times, the total number of statements for each loop must be at least 10, any loops or scans from the input file will not be expanded, and vectors can be compressed despite having comments between them.

4.2.10 Merge Files

The '*merge_files*' command is designed to merge STIL patterns from the specified files into the processed file, creating serial or parallel structures.

Parameter	Req'd	Value	Description
files	Yes	<i>list</i>	STIL files to merge.
merge_mode	No	serial / parallel	Merge mode. Default is serial.
merge_patterns	No	<i>boolean</i>	Merge patterns flag. Default is 0. If 1, merge patterns into one. Otherwise combine patterns using PatternBurst.
unify_periods	No	<i>boolean</i>	Apply unify periods. Default is 0.
max_wfts	No	<i>integer</i>	Max allowed number of WFTs. Default is 0 meaning unlimited.
allow_new_waveforms	No	<i>boolean</i>	Allow new waveforms flag. Default is 1. If 1, new waveforms can be added to signals.

allow_wfc_remap	No	<i>boolean</i>	Allow WFC remap flag. Default is 1. If 1, conflicting waveforms will be remapped to new WFCs and added to signals.
use_file_name_as_pat	No	<i>boolean</i>	Use file name as pattern name flag. Default is 1. If 1, use STIL file name for output pattern.
min_period	No	<i>string</i>	Min allowed period if unify periods is applied. Default is min input period divided by 10000.
inputs_padding	No	0/1/last	Padding state for inputs. Default is last state.
outputs_padding	No	L/H/T/X	Padding state for outputs. Default is X.

Example

```
merge_files(
    files           = ['file2.stil','file3.stil'],
    merge_mode      = 'parallel',
    merge_patterns  = 1,
    unify_periods   = 1,
    max_wfts        = 4,
    allow_new_waveforms = 1,
    allow_wfc_remap = 1,
    use_file_name_as_pat = 1,
    min_period      = '100ns',
    inputs_padding  = 'last',
    outputs_padding = 'X'
)
```

Pattern Merge Configuration Options

To explain the way merge files command works, let's consider the following files.

STIL Sample 4.1: P1.stil

```
1 PatternBurst B1 {
2   PatList {
3     P1;
4   }
5 }
6
7
8 Pattern P1 {
9   W W1;
10  V { A0 = 0; A1 = 0; }
11  V { A0 = 1; A1 = 0; }
12 }
```

STIL Sample 4.2: P2.stil

```
1 PatternBurst B1 {
2   PatList {
3     P2;
4   }
5 }
6
7 Pattern P2 {
8   W W1;
9   V { A0 = 1; A1 = 1; }
10  V { A0 = 1; A1 = 0; }
11  V { A0 = 1; A1 = 1; }
12 }
```

STIL Sample 4.3: P3.stil

```
1 PatternBurst B1 {
2   PatList {
3     P3;
4   }
5 }
6
7
8 Pattern P3 {
9   W W1;
10  V { B0 = 1; B1 = 1; }
11  V { B0 = 0; B1 = 1; }
12 }
```

P1 will be STILEditor input file. P2 or P3 will be merged into it using the merge_files command, e.g.

Example

```
merge_files(
    files           = ['P2.stil'],
    merge_mode      = 'serial',
    merge_patterns  = 0,
)
```

There are two parameters defining the pattern merge process - 'merge_mode' and 'merge_patterns'. Results for different combinations of these parameters are presented below.

'merge_mode' parameter allows choice between serial and parallel merge.

In order to make an in-pattern merging and get a single pattern, enable the 'merge_patterns' flag. By default the 'merge_patterns' parameter is disabled, so the patterns are passed as is to the output, and the PatList structure ('serial' or 'parallel') is set in the burst block. When the merge_patterns flag is enabled, all patterns must contain disjoint sets of signals in parallel merge mode, and identical sets of signals in serial merge mode.

STIL Sample 4.4: Merge Pattern Off, Serial, P2 into P1

```
1 PatternBurst "merge_burst" {
2   PatList {
3     "P1";
4     "P2";
5   }
6 }
7
8 Pattern "P1" {
9   W "W1";
10  V { "A0"=0; "A1"=0; }
11  V { "A0"=1; }
12 }
13
14
15 Pattern "P2" {
16   W "W1";
17   V { "A0"=1; "A1"=1; }
18   V { "A1"=0; }
19   V { "A1"=1; }
20 }
```

STIL Sample 4.5: Merge Pattern On -Serial(P1,P2)

```
1 PatternBurst "merge_burst" {
2   PatList {
3     "master";
4   }
5 }
6
7 Pattern "master" {
8   // File: P1.stil
9   // Pattern: P1
10  "P1":
11  W "W1";
12  V { "A0"=0; "A1"=0; }
13  V { "A0"=1; }
14  // File: P2.stil
15  // Pattern: P2
16  "P2":
17  V { "A1"=1; }
18  V { "A1"=0; }
19  V { "A1"=1; }
20 }
```

STIL Sample 4.6: Merge Pattern Off -Parallel(P1,P3)

```
1 PatternBurst "merge_burst" {
2   ParallelPatList SyncStart
3   {
4     "P1";
5     "P3";
6   }
7 }
8
9 Pattern "P1" {
10  W "W1";
11  V { "A0"=0; "A1"=0; }
12  V { "A0"=1; }
13 }
14
15 Pattern "P3" {
16  W "W1";
17  V { "B0"=1; "B1"=1; }
18  V { "B0"=0; }
19 }
```

STIL Sample 4.7: Merge Pattern On -Parallel(P1,P3)

```
1 PatternBurst "merge_burst" {
2   PatList {
3     "master";
4   }
5 }
6
7
8
9 Pattern "master" {
10  W "W1";
11  V { "A0"=0; "A1"=0; "B0"
12    =1; "B1"=1; }
13  V { "A0"=1; "B0"=0; }
14 }
```

Timing Merge Configuration Options

Parameters 'allow_new_waveforms', 'allow_wfc_remap', 'max_wfts' and 'unify_periods' control the way input timings are merged.

If a subsequent STIL file contains waveforms which didn't exist in the earlier STIL files, by default STILEditor adds new waveforms as needed. If the 'allow_new_waveforms' flag is disabled, error will be issued instead.

Conflicting waveforms are waveforms with identical WFC but different content. By default STILEditor resolves these conflicts by remapping a waveform to a new WFC. If 'allow_wfc_remap' is disabled, an error will be issued when remapping is necessary.

In order to limit the max allowed number of WFTs, define max number for 'max_wfts' option. If this limitation is violated, an error will be issued. If set to '0' (default), the number of WFTs is not limited.

If 'unify_periods' flag is enabled, STILEditor will convert the source WaveformTables to a single WaveformTable, similarly to 'unify_wft_period' (see [4.2.7](#)). This flag is useful for situations where the input test contains more periods/WaveformTables than the upper limit of specific targets. The output WaveformTable period will be the GCD (greatest common divisor) of the original periods. In case the calculated GCD is different than the period of a certain WFT, the period of that WFT will be changed to the GCD value, and the number of vectors in the patterns using that WFT will be changed accordingly to fit the original pattern execution time. Note that the 'unify_periods' parameter is configurable only in the serial merge mode (see [4.2.10](#)); the default is off. In parallel merge mode, unification is applied automatically.

The 'min_period' parameter sets minimal allowed cycle width. If unify period results in cycle width smaller than the specified value, STILEditor stops with error. Default is minimal input period divided by 10000.

Pattern Padding

It is possible to determine the signal values in cases where padding is required, i.e. when we choose parallel merge between patterns with different lengths. By default, input values are padded with their last value, and output values are padded with X. Set 'inputs_padding' or 'outputs_padding' to determine the padding values.

Other Parameters

Pattern naming convention flag (optional).

If 0, STIL Pattern name is used when generating pattern.

If 1 (default), STIL File name is used as the pattern name (for a single pattern) or as the pattern name prefix (for multiple patterns).

Example

```
stil_file_name_as_pattern_name = 1
```

4.2.11 Merge Bursts

The 'merge_bursts' command merges complex burst into a single pattern. For the main burst and each of its sub-bursts serial or parallel merge (see [4.2.10](#)) is selected according to the PatList type.

The 'min_period' parameter sets minimal allowed cycle width. If unify period results

in cycle width smaller than the specified value, STIEditor stops with error. Default is minimal input period divided by 10000.

Parameter	Req'd	Value	Description
min_period	No	<i>string</i>	Min allowed period if unify period is applied. Default is min input period divided by 10000. If unify period results in cycle width smaller than the specified value, STIEditor stops with error.
pat_name	No	<i>string</i>	Output pattern name. Default is 'PATTERN'.

Example

```
merge_bursts(
    min_period      = '100ns',
    pat_name        = 'merged_pattern',
)
```

4.2.12 Split Pattern To Ports

'*split_pattern_to_ports*' command splits an input pattern to a set of parallel patterns. For each port, a separate pattern is created, containing signals specified for that port.

Parameter	Req'd	Value	Description
pat_name	No	<i>string</i>	Pattern name. If empty, all patterns are processed.
ports	Yes	<i>dictionary</i>	Ports definitions. Format is port1 : signals1, port2 : signals2,

Example

```
split_pattern_to_ports(
    pat_name = '',
    ports    = {
        'port1': [ 'sig1', 'sig2' ],
        'port2': [ 'sig3', 'sig4' ]
    }
)
```

4.2.13 Other Commands

Import Blocks

'*import_blocks*' command allows to import STIL blocks from an external STIL file to the STIEditor input file.

Parameter	Req'd	Value	Description
path	Yes	<i>string</i>	Path to STIL file containing the required blocks.
import_signals	No	<i>boolean</i>	If 1, STIL Signals will be imported.
import_groups	No	<i>boolean</i>	If 1, STIL SignalGroups will be imported.
import_specs	No	<i>boolean</i>	If 1, STIL Specs will be imported.
import_timing	No	<i>boolean</i>	If 1, STIL Timing will be imported.
import_dclevels	No	<i>boolean</i>	If 1, STIL DCLevels will be imported.

import_env	No	<i>boolean</i>	If 1, STIL Environment will be imported.
update_exec	No	<i>boolean</i>	If 1, imported dclevels/categories/selectors will be added to the active PatternExec.

Example

```
import_blocks(
    path           = 'external.stil',
    import_signals = 1,
    import_groups  = 1,
    import_specs   = 1,
    import_timing  = 1,
    import_dclevels = 1,
    import_env     = 1,
    update_exec    = 1
)
```

Add Free Running Clock

'add_free_run_clk' command adds a free running clock definition to the SignalGroups STIL block.

If the workdir parameter is not specified, the file will be saved at the same directory as the global output directory. The Name of the file is composed of the prefix parameter value and the original file name.

Parameter	Req'd	Value	Description
name	Yes	<i>string</i>	New signal name.
period	Yes	<i>string</i>	Clock period.
lead_edge	Yes	<i>string</i>	Leading edge.
trail_edge	Yes	<i>string</i>	Trailing edge.
off_state	Yes	D/U/N/Z/L/H/X/T	Off state.

Example

```
add_free_run_clk(
    name      = 'clk',
    period    = '100ns',
    lead_edge = '0ns',
    trail_edge = '20ns',
    off_state = 'D'
)
```

Write

The 'write' command saves the intermediate results after the execution of all the commands that preceded this command.

Parameter	Req'd	Value	Description
workdir	No	<i>string</i>	Output directory. Default is global workdir.
prefix	No	<i>string</i>	File name prefix. Default is ". Output file name will be composed from the prefix and the original file name.

Example

```
write(  
    workdir = 'intermediate_results',  
    prefix  = 'temp_'  
)
```

4.3 Multiple Commands Spreadsheet File

STIEditor can use an external tabular file (XLSX or CSV) to perform multiple signal manipulations commands (see 4.2.1) more conveniently. For these cases, the user may use the same signal manipulations commands followed by a '_file' extension in the setup file:

- rename_signals_file
- delete_signals_file
- add_signals_fixed_file
- add_signals_copy_file

All the above commands use the same parameters:

Parameter	Req'd	Value	Description
path	Yes	<i>string</i>	Location of the external file.
sheet_name	No	<i>string</i>	Sheet name in the external file. Only applicable if XLSX file is used.

The commands will be arranged in a table shape, **aligned to the left end of the sheet**, so each line represents a single command. The rows are to be filled according to the command parameters. In the following example, an external file is used to add 2 fixed signals to the test. Assume the file is named 'input.xlsx', and the sheet is named 'fixed_signals'

Setup Command

```
add_signals_fixed_file(  
    path      = 'input.xlsx',  
    sheet_name = 'fixed_signals'  
)
```

Table 4.35: Content of fixed_signal sheet in file input.xlsx

Pin1	In	U	20ns
Pin2	InOut	Z	40ns

Equivalent Single Step Setup Commands

```
add_signal_fixed(  
    name      = 'Pin1',  
    sig_type  = 'In',  
    state     = 'U',  
    time      = '20ns'  
)  
add_signal_fixed(  
    name      = 'Pin2',  
    sig_type  = 'InOut',  
    state     = 'Z',  
    time      = '40ns'  
)
```


Chapter 5

Using STILEditor in Import Pins Mode

Import pins mode of STILEditor is designed to create a pin usage table in a CSV file. The user should provide a path to one or more STIL files, which STILEditor will use to create the table, based on the patterns and the pins used in these files. Import pins mode does not use a setup file as a command line argument.

5.1 Example of import_pins Usage for a Single File

```
> stileditor -import_pins filename.stil
```

STIL Sample 5.1: Single Input File

```
1 PatternBurst B1 {  
2   PatList {  
3     P1;  
4     P2;  
5   }  
6 }  
7 Signals {  
8   clk In;  
9   pin1 In;  
10  pin2 Out;  
11 }  
12 Pattern P1 {  
13   W t1;  
14   V {clk = P; pin1 = 1;}  
15 }  
16 Pattern P2 {  
17   W t1;  
18   V {clk = P; pin1 = 0;}  
19   V {pin2 = L;}  
20 }
```

In the output CSV file, the first column denotes the pin names, the second denotes the type of the pin, and each of the next columns represents the patterns used in the input files. A cell will hold the value Y if the pin in the current row is used at least once in the pattern of the current column, or N otherwise.

#Pin Name	Pin Type	Active [P1]	Active [P2]
clk	In	Y	Y
pin1	In	Y	Y
pin2	Out	N	Y

Table 5.1: Resulting CSV file

5.2 Example of import_pins Usage for Multiple Files

STIEditor may also generate a combined signal usage table for multiple files using the CLI.

```
> stileditor -import_pins filename1.stil filename2.stil
```

STIL Sample 5.2: Import Pins Input 1

```

1 PatternBurst B1 {
2   PatList {
3     P1;
4     P2;
5   }
6 }
7 Signals {
8   clk In;
9   pin1 In;
10  pin2 Out;
11 }
12 Pattern P1 {
13   W t1;
14   V {clk = P; pin1 = 0; }
15 }
16 Pattern P2 {
17   W t1;
18   V {clk = P; }
19   V {pin2 = L; }
20 }
```

STIL Sample 5.3: Import Pins Input 2

```

1 PatternBurst B1 {
2   PatList {
3     P2;
4     P3;
5   }
6 }
7 Signals {
8   clk In;
9   pin2 Out;
10  pin3 In;
11 }
12 Pattern P2 {
13   W t1;
14   V {clk = P; pin3 = 0;}
15 }
16 Pattern P3 {
17   W t1;
18   V {clk = P; pin3 = 1;}
19   V {pin2 = L;}
20 }
```

When using STIEditor to import pins from multiple files, the output CSV file will combine the usage information of pins or patterns which appear in more than one file, into a single row or column respectively. Only a single CSV file is created with import_pins, so there might be intersecting cells between signals and patterns they don't appear in, but the value in these cells will always be N.

#Pin Name	Pin Type	Active [P1]	Active [P2]	Active [P3]
clk	In	Y	Y	Y
pin1	In	Y	N	N
pin2	Out	N	Y	Y
pin3	In	N	Y	Y

Table 5.2: Resulting CSV file

Chapter 6

Appendix

6.0.1 STIEditor Global Configuration

- Input Files (Mandatory if not specified in CLI). Alternative STIL files definition. CLI definition always overwrites this one, if any.

Example

```
input_files = ['input1.stil','input2.stil']
```

- STIEditor output directory (optional). Default value is 'output'. CLI definition always overwrites this one, if any.

Example

```
workdir = './output'
```

- Show STIL statistics (optional).

Example

```
show_info = 1
```

6.0.2 STIL Writer Customization

- STIL pattern formatting mode. Valid values are: 'write_on_change' or 'write_as_group'. default is 'write_on_change'.

Example

```
stil_pattern_mode = 'write_on_change'
```

- Write timing per pin (optional). Split timings into separate individual signals.

Example

```
stil_timing_per_pin = 1
```

- Write DC levels per pin (optional). Split levels into separate individual signals.

Example

```
stil_dclevels_per_pin = 1
```
